Resumen: Lenguaje Imperativo Simple (LIS)

1. Lenguaje Imperativo Simple (LIS)

1.1. Gramática abstracta

$$
\begin{array}{lcl}
\langle comm \rangle & ::= & \textbf{skip} \\
 & & \langle var \rangle := \langle intexp \rangle \\
 & & \langle comm \rangle \ ; \ \langle comm \rangle \\
 & & \textbf{if} \ \langle boolexp \rangle \ \textbf{then} \ \langle comm \rangle \ \textbf{else} \ \langle comm \rangle \\
 & & \textbf{newvar} \ \langle var \rangle := \langle intexp \rangle \ \textbf{in} \ \langle comm \rangle \\
 & & \textbf{while} \ \langle boolexp \rangle \ \textbf{do} \ \langle comm \rangle
\end{array}
$$

$$
\begin{array}{lcl}
\langle intexp \rangle & ::= & \langle natconst \rangle \\
 & & \langle var \rangle \\
 & & \langle intexp \rangle + \langle intexp \rangle \\
 & & \langle intexp \rangle * \langle intexp \rangle \\
 & & \langle intexp \rangle - \langle intexp \rangle \\
 & & \langle intexp \rangle \ / \ \langle intexp \rangle \\
 & & \langle intexp \rangle \ \% \ \langle intexp \rangle \\
 & & - \langle intexp \rangle
\end{array}
$$

$$
\begin{array}{lcl}
\langle boolexp \rangle & ::= & \langle boolconst \rangle \\
 & & \langle intexp \rangle = \langle intexp \rangle \\
 & & \langle intexp \rangle < \langle intexp \rangle \\
 & & \langle intexp \rangle \leq \langle intexp \rangle \\
 & & \langle intexp \rangle > \langle intexp \rangle \\
 & & \langle intexp \rangle \geq \langle intexp \rangle \\
 & & \neg \langle boolexp \rangle \\
 & & \langle boolexp \rangle \vee \langle boolexp \rangle \\
 & & \langle boolexp \rangle \wedge \langle boolexp \rangle
\end{array}
$$

$$
\begin{array}{lcl}
\langle natconst \rangle & ::= & 0 \mid 1 \mid 2 \mid .... \\
\langle boolconst \rangle & ::= & \textbf{true} \mid \textbf{false}
\end{array}
$$

1.2. Estados

Una función que asigna a cada variable un entero $\Sigma = \text{Var} \to \mathbb{Z}$

2. Funciones semánticas

$$
\begin{aligned}
[\![\cdot]\!]_{intexp} &: \langle intexp \rangle \to (\Sigma \to \mathbb{Z}) \\
[\![\cdot]\!]_{boolexp} &: \langle boolexp \rangle \to (\Sigma \to \{V, F\}) \\
[\![\cdot]\!]_{comm} &: \langle comm \rangle \to (\Sigma \to \Sigma)
\end{aligned}
$$

Todos los comandos terminan y dan un nuevo estado

3. No terminación

Se introduce: $\Sigma_\perp = \Sigma \cup \{\perp\}$. Así que: $[\![\cdot]\!] : \langle comm \rangle \to (\Sigma \to \Sigma_\perp)$

4. Extensión estricta

Dada una $f \in P \to D$, donde P es un predominio y D un dominio, se denota por $f_{\perp\perp}$ la única extensión estricta de f a $P_\perp$. Así, $f_{\perp\perp} \in P_\perp \to D$ está definida por:

$$f_{\perp\perp}\, x = \begin{cases} \perp & \text{si } x = \perp \\ f\, x & \text{si no} \end{cases}$$

5. Resumen de llaves semanticas

Expresiones Enteras

$[\![\cdot]\!] \in \langle intexp \rangle \to \Sigma \to \mathbb{Z}$

$$\begin{aligned} [\![0]\!]\sigma &= 0 \\ [\![-e]\!]\sigma &= -[\![e]\!]\sigma \\ [\![e_0 + e_1]\!]\sigma &= [\![e_0]\!]\sigma + [\![e_1]\!]\sigma \quad \text{(similarmente para } -, *, \dots) \\ [\![v]\!]\sigma &= \sigma\, v \end{aligned}$$

Expresiones Booleanas

$[\![\cdot]\!] \in \langle boolexp \rangle \to \Sigma \to \mathbb{B}$

$$\begin{aligned} [\![\text{true}]\!]\sigma &= V \\ [\![\text{false}]\!]\sigma &= F \\ [\![e_0 = e_1]\!]\sigma &= [\![e_0]\!]\sigma = [\![e_1]\!]\sigma \quad \text{(similarmente para } <, >, \dots) \\ [\![\neg b]\!]\sigma &= \neg [\![b]\!]\sigma \\ [\![b_0 \wedge b_1]\!]\sigma &= [\![b_0]\!]\sigma \wedge [\![b_1]\!]\sigma \quad \text{(similarmente para } \vee, \dots) \end{aligned}$$

Comandos

$[\![\cdot]\!] \in \langle comm \rangle \to \Sigma \to \Sigma_\perp$

$$
\begin{aligned}
[\![\text{skip}]\!]\sigma &= \sigma \\
[\![v := e]\!]\sigma &= [\sigma | v : [\![e]\!]\sigma] \\
[\![\text{if } b \text{ then } c_0 \text{ else } c_1]\!]\sigma &= \begin{cases} [\![c_0]\!]\sigma & \text{si } [\![b]\!]\sigma \\ [\![c_1]\!]\sigma & \text{si no} \end{cases} \\
[\![c_0; c_1]\!]\sigma &= [\![c_1]\!]_\perp([\![c_0]\!]\sigma) \\
[\![\text{newvar } v := e \text{ in } c]\!]\sigma &= (\lambda\sigma' \in \Sigma.[\sigma' | v : \sigma\, v])_\perp([\![c]\!][\sigma | v : [\![e]\!]\sigma]) \\
[\![\text{while } b \text{ do } c]\!] &= \bigsqcup_{i=0}^{\infty} F^i \perp_{\Sigma \to \Sigma_\perp}
\end{aligned}
$$

donde:

$$
\begin{aligned}
f_{\perp\perp} x &= \begin{cases} \perp & \text{si } x = \perp \\ f\, x & \text{si no} \end{cases} \\
F\, w\, \sigma &= \begin{cases} w_{\perp\perp}([\![c]\!]\sigma) & \text{si } [\![b]\!]\sigma \\ \sigma & \text{si no} \end{cases}
\end{aligned}
$$

6. Propiedades

6.1. Variables libres

$$
\begin{aligned}
FV(\texttt{skip}) &= \emptyset \\
FV(v := e) &= \{v\} \cup FV(e) \\
FV(c_0; c_1) &= FV(c_0) \cup FV(c_1) \\
FV(\texttt{if } b \texttt{ then } c_0 \texttt{ else } c_1) &= FV(b) \cup FV(c_0) \cup FV(c_1) \\
FV(\texttt{while } b \texttt{ do } c) &= FV(b) \cup FV(c) \\
FV(\texttt{newvar } v := e \texttt{ in } c) &= FV(e) \cup (FV(c) - \{v\})
\end{aligned}
$$

6.2. Variables asignables

$$
\begin{aligned}
FA(\texttt{skip}) &= \emptyset \\
FA(v := e) &= \{v\} \\
FA(c_0; c_1) &= FA(c_0) \cup FA(c_1) \\
FA(\texttt{if } b \texttt{ then } c_0 \texttt{ else } c_1) &= FA(c_0) \cup FA(c_1) \\
FA(\texttt{while } b \texttt{ do } c) &= FA(c) \\
FA(\texttt{newvar } v := e \texttt{ in } c) &= FV(c) - \{v\}
\end{aligned}
$$

6.3. Teorema de Coincidencia

1. $(\forall w \in FV(c).\sigma w = \sigma' w)$ implica,
   - o bien $[\![c]\!]\sigma = \bot = [\![c]\!]\sigma'$,
   - o bien $[\![c]\!]\sigma \neq \bot \neq [\![c]\!]\sigma'$ y $\forall w \in FV(c).[\![c]\!]\sigma w = [\![c]\!]\sigma' w$
2. Si $[\![c]\!]\sigma \neq \bot$, entonces $\forall w \notin FA(c).[\![c]\!]\sigma w = \sigma w$. (esto es intangibilidad)

6.4. Teorema de Renombre

Otra propiedad que cobra importancia en el lenguaje imperativo es la que establece que no importa el nombre de las variables utilizadas en las declaraciones de variables locales (o sea las ligadas):

$$u \notin FV(c) - \{v\} \Rightarrow [\![\textbf{newvar } u := e \textbf{ in } c[v \to u]]\!] = [\![\textbf{newvar } v := e \textbf{ in } c]\!]$$

7. Sustituciones

Conjunto de sustituciones

$$\Delta = \mathrm{Var} \to \mathrm{Var}$$

Las ecuaciones que la definían para la lógica de predicados siguen valiendo (salvo el caso de los cuantificadores, que ahora no tenemos). Ahora se agregan ecuaciones para los comandos.

$$_/_ \in \langle comm \rangle \times \Delta \to \langle comm \rangle$$

$$\begin{aligned}
\textbf{skip}/\delta &= \textbf{skip} \\
(v := e)/\delta &= (\delta v) := (e/\delta) \\
(c_0; c_1)/\delta &= (c_0/\delta); (c_1/\delta) \\
(\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1)/\delta &= \textbf{if } b/\delta \textbf{ then } c_0/\delta \textbf{ else } c_1/\delta \\
(\textbf{while } b \textbf{ do } c)/\delta &= \textbf{while } b/\delta \textbf{ do } c/\delta
\end{aligned}$$

El caso del **newvar** tiene los mismos inconvenientes que los cuantificadores, ahora es un poco más simple ya que δw son siempre variables.

$$(\textbf{newvar } v := e \textbf{ in } c)/\delta = \textbf{newvar } v_{new} := e/\delta \textbf{ in } (c/[\delta|v : v_{new}])$$

donde $v_{new} \notin \{\delta w | w \in FV(c) - \{v\}\}$

7.1. Teorema de Sustitución

Si δ es inyectiva sobre $FV(c)$ y $\forall w \in FV(c).\ \sigma(\delta w) = \sigma' w$, entonces

- o bien $[\![c/\delta]\!]\sigma = \bot = [\![c]\!]\sigma'$,
- o bien $[\![c/\delta]\!]\sigma \neq \bot \neq [\![c]\!]\sigma'$ y $\forall w \in FV(c).\ [\![c/\delta]\!]\sigma(\delta w) = [\![c]\!]\sigma' w$.

7.2. Lema de Sustitución

Sea $FV(c) \subseteq V \subseteq \langle var \rangle$ tal que δ es inyectiva sobre V y $\forall w \in V.\ \sigma(\delta w) = \sigma' w$. Entonces,

- o bien $[\![c/\delta]\!]\sigma = \bot = [\![c]\!]\sigma'$,
- o bien $[\![c/\delta]\!]\sigma \neq \bot \neq [\![c]\!]\sigma'$ y $\forall w \in V.\ [\![c/\delta]\!]\sigma(\delta w) = [\![c]\!]\sigma' w$.

8. Fallas

Para modelar la transferencia de control por fallas, se incorporan:

$$\langle comm \rangle ::= fail \mid catchin\ \langle comm \rangle\ with\ \langle comm \rangle$$

Se extiende: $\Sigma' = \Sigma \cup (\{abort\} \times \Sigma)$ Y la función semántica queda definida como: $[[_]] : \langle comm \rangle \to (\Sigma \to \Sigma'_\bot)$ donde $\Sigma'_\bot$ es un dominio llano con elemento indefinido $\bot$.

8.1. Operadores de transferencia de control

Para definir la semántica de composición y manejo de excepciones, se introducen tres operadores:

- **Extensión estricta (f_*):** Propaga fallas (no continúa la ejecución).

$$f_*(x) = \begin{cases} f(\sigma) & \text{si } x = \sigma \in \Sigma \\ x & \text{si } x \text{ es abort o } \bot \end{cases}$$

- **Extensión de manejo (f_+):** Se ejecuta solo si hubo falla.

$$f_+(x) = \begin{cases} f(\sigma) & \text{si } x = \langle abort, \sigma \rangle \\ x & \text{en otro caso} \end{cases}$$

- **Extensión total ($f_\dagger$):** Se ejecuta siempre, preservando el estado de falla (en este caso se hace asi ya que queremos que en el newvar se ejecute la función para restaurar el estado original).

$$f_\dagger(x) = \begin{cases} \langle abort, f(\sigma) \rangle & \text{si } x = \langle abort, \sigma \rangle \\ f(x) & \text{si } x \in \Sigma \\ \bot & \text{si } x = \bot \end{cases}$$

8.2. Semántica final

$$\begin{aligned}
[[skip]]\sigma &= \sigma \\
[[fail]]\sigma &= \langle abort, \sigma \rangle \\
[[v := e]]\sigma &= \sigma[v \mapsto [[e]]\sigma] \\
[[c_0; c_1]]\sigma &= [[c_1]]_*([[c_0]]\sigma) \\
[[catchin\ c_0\ with\ c_1]]\sigma &= [[c_1]]_+([[c_0]]\sigma) \\
[[newvar\ v := e\ in\ c]]\sigma &= (\lambda\sigma'.\sigma'[v \mapsto \sigma(v)])_\dagger([[c]](\sigma[v \mapsto [[e]]\sigma])) \\
[[while\ b\ do\ c]] &= \bigsqcup_{i=0}^{\infty} F^i(\bot)
\end{aligned}$$

donde

$$F(w)(\sigma) = \begin{cases} w_*([[c]]\sigma) & \text{si } [[b]]\sigma \\ \sigma & \text{en otro caso} \end{cases}$$

9. Entrada/Salida

$$\langle comm \rangle \quad ::= \quad \text{skip} \mid ... \mid !\langle intexp \rangle \mid ?\langle var \rangle$$

El dominio de salidas Ω se define como el conjunto de posibles resultados de la ejecución de un programa, incluyendo terminación exitosa, error o divergencia (ejecución infinita).

$$\begin{aligned} \Omega = \quad & \{\langle n_0, \ldots, n_k, \sigma' \rangle \mid n_i \in \mathbb{Z}, \sigma' \in \hat{\Sigma}\} \\ & \cup \{\langle n_0, \ldots, n_k \rangle \mid n_i \in \mathbb{Z}\} \\ & \cup \{\langle n_0, n_1, n_2, n_3, \ldots, n_k, \ldots \rangle \mid n_i \in \mathbb{Z}\} \end{aligned}$$

Donde el conjunto de estados extendido $\hat{\Sigma}$ se define como:

$$\hat{\Sigma} = \Sigma \cup (\{\mathbf{abort}\} \times \Sigma)$$

9.1. Concatenación de tuplas

Podemos "concatenar" ciertos elementos de Ω para obtener nuevos elementos del mismo dominio:

$$\langle n_0, \ldots, n_k \rangle +\!\!+ \langle m_0, \ldots, s \rangle = \langle n_0, \ldots, n_k, m_0, \ldots, s \rangle$$

Aquí, s puede representar tanto un número entero como un estado (finalizado o fallido).

9.2. Estructura del Dominio

Para tratar la semántica de forma denotacional, definimos una estructura de orden sobre Ω:

1. **Orden de prefijo:** El orden está dado por la relación de prefijo. Por ejemplo:
$$\langle 4, 3, 2, 1 \rangle \sqsubseteq \langle 4, 3, 2, 1, \sigma \rangle$$

2. **Supremo de cadenas no interesantes:** Es simplemente el último elemento de la cadena.

3. **Supremo de cadenas interesantes:** Es la tupla (posiblemente infinita) que coincide en todas las posiciones de cada tupla de la cadena.

Observaciones:

- El elemento mínimo está dado por la tupla vacía: $\langle \rangle$.
- En una cadena "interesante" de tuplas, existen tuplas de cada longitud, por lo tanto, el supremo es efectivamente único.

9.3. Semántica de Programas de output

$[\![_]\!] : \langle comm \rangle \to (\Sigma \to \Omega)$

$$[\![!e]\!]\sigma = \langle [\![e]\!]\sigma, \sigma \rangle$$

$$[\![\mathbf{skip}]\!]\sigma = \langle \sigma \rangle$$

$$[\![v := e]\!]\sigma = \langle \sigma[v : [\![e]\!]\sigma] \rangle$$

$$[\![\mathbf{if}\ b\ \mathbf{then}\ c\ \mathbf{else}\ c']\!]\sigma = \begin{cases} [\![c]\!]\sigma & \text{si } [\![b]\!]\sigma \\ [\![c']\!]\sigma & \text{si } \neg[\![b]\!]\sigma \end{cases}$$

$$[\![c_0; c_1]\!]\sigma = [\![c_1]\!]_*([\![c_0]\!]\sigma)$$

$$[\![\mathbf{newvar}\ v := e\ \mathbf{in}\ c]\!]\sigma = (rest_{v,\sigma})_\dagger\,([\![c]\!][\sigma \mid v : [\![e]\!]\sigma])$$

$$F_{b,c} : (\Sigma \to \Sigma') \to (\Sigma \to \Sigma')$$

$$F_{b,c}(f)(\sigma) = \begin{cases} \sigma & \text{si } \neg[\![b]\!]\sigma \\ f_*([\![c]\!]\sigma) & \text{si } [\![b]\!]\sigma \end{cases}$$

$$[\![\mathbf{while}\ b\ \mathbf{do}\ c]\!]\sigma = \left(\bigsqcup_{i \in \mathbb{N}} F^i\ \bot \right) \sigma$$

10. Funciones de Propagación de Errores

Estas funciones definen cómo se comportan las transformaciones de estado cuando encuentran estados de error o terminación.

10.1. Propagación estándar $(_)_*$

La función $(_)_*$ extiende una función de estado a una que maneja errores y abortos:

$$(_)_* : (\Sigma \to \Sigma') \to (\Sigma' \to \Sigma')$$

$$f_*(\bot) = \bot$$
$$f_*(\langle \mathbf{abort}, \sigma \rangle) = \langle \mathbf{abort}, \sigma \rangle$$
$$f_*(\sigma) = f(\sigma)$$

10.2. Propagación tipo Map $(_)_\dagger$

Similar a la función `map` en listas, aplica una transformación incluso en estados de aborto:

$$(_)_\dagger : (\Sigma \to \Sigma) \to (\Sigma' \to \Sigma')$$

$$\begin{aligned} f_\dagger(\bot) &= \bot \\ f_\dagger(\langle \mathbf{abort}, \sigma \rangle) &= \langle \mathbf{abort}, f(\sigma) \rangle \\ f_\dagger(\sigma) &= f(\sigma) \end{aligned}$$

11. Revisión de Funciones sobre Ω

Al trabajar con el dominio de secuencias Ω, las funciones de propagación se redefinen para operar sobre tuplas de enteros y estados.

11.1. Propagación en Ω

$$(_)_* : (\Sigma \to \Omega) \to (\Omega \to \Omega)$$

$$\begin{aligned} f_*(\langle n_0, \ldots, n_{k-1} \rangle) &= \langle n_0, \ldots, n_{k-1} \rangle \\ f_*(\langle n_0, \ldots, n_{k-1}, \sigma \rangle) &= \langle n_0, \ldots, n_{k-1} \rangle \mathbin{+\!\!+} f(\sigma) \\ f_*(\langle n_0, \ldots, n_{k-1}, \langle \mathbf{abort}, \sigma \rangle \rangle) &= \langle n_0, \ldots, n_{k-1}, \langle \mathbf{abort}, \sigma \rangle \rangle \\ f_*(\langle n_0, \ldots, n_{k-1}, \ldots \rangle) &= \langle n_0, \ldots, n_{k-1}, \ldots \rangle \end{aligned}$$

11.2. Función Map en Ω

$$(_)_\dagger : (\Sigma \to \Sigma) \to (\Omega \to \Omega)$$

$$\begin{aligned} f_\dagger(\langle n_0, \ldots, n_{k-1} \rangle) &= \langle n_0, \ldots, n_{k-1} \rangle \\ f_\dagger(\langle n_0, \ldots, n_{k-1}, \sigma \rangle) &= \langle n_0, \ldots, n_{k-1}, f(\sigma) \rangle \\ f_\dagger(\langle n_0, \ldots, n_{k-1}, \langle \mathbf{abort}, \sigma \rangle \rangle) &= \langle n_0, \ldots, n_{k-1}, \langle \mathbf{abort}, f(\sigma) \rangle \rangle \\ f_\dagger(\langle n_0, \ldots, n_{k-1}, \ldots \rangle) &= \langle n_0, \ldots, n_{k-1}, \ldots \rangle \end{aligned}$$

12. Funciones Inyectoras y Constructores

En semántica denotacional, las funciones iota (ι) son piezas fundamentales para construir y navegar por dominios compuestos. Que nos permiten meter un valor simple dentro de una estructura mucho más compleja (una unión disjunta) sin que se pierda su identidad.

En términos técnicos, se llaman inyecciones canónicas, y su función principal es el ajuste de tipos para que el lenguaje formal sea coherente.

1. El contenedor universal (Ω) Imagina que el dominio Ω (el resultado de tu programa) es una caja que puede guardar cosas de naturalezas muy distintas:

- Un estado de memoria (un diccionario de variables).
- Un mensaje de error (un simple .ªbort").
- Un número que sale por pantalla.
- Una instrucción que espera que el usuario escriba algo.

Matemáticamente, no podes "sumarün número con un diccionario. Sería como intentar sumar 5 peras + "Hola". Las funciones ι envuelven a cada elemento en un **empaque estándar** para que todos "midan"lo mismo y puedan vivir dentro de Ω.

2. Etiquetas invisibles Lo más importante de las iotas es que no solo mueven el valor, sino que le pegan una etiqueta invisible.

- $\iota_{norm}(\sigma)$ le dice al sistema: .ᴱˢto es un estado normal".
- $\iota_{abnorm}(\sigma)$ le dice al sistema: .ᴱˢto es un estado de error".

Sin la iota, verías solo el estado σ y no sabrías si es un resultado exitoso o un error. La iota permite que, más adelante, otras funciones puedan preguntar: ¿Qué etiqueta trae este valor? y actuar en consecuencia.

3. ¿Por qué es necesario compararlos? Se necesitan en el mismo espacio para poder operar con ellos. En la semántica de comandos compuestos (como 'c1 ; c2'): 1. El comando 'c1' devuelve algo en el espacio Ω. 2. El comando 'c2' necesita saber si lo que devolvió 'c1' fue un estado normal para seguir ejecutando, o si fue un error para detenerse.

Si ambos resultados no estuvieran en el mismo espacio (Ω), no podrías definir una sola función que maneje ambas posibilidades. Las iotas transforman la diversidad de resultados en un tipo de dato uniforme.

$$[\![?v]\!]\sigma \quad = \quad \iota_{in}(\lambda n \in \mathbf{Z}.\ \iota_{term}[\sigma|v : n])$$

$$f_* x = \begin{cases} \bot_\Omega & x = \bot_\Omega \\ f\sigma & x = \iota_{term}\sigma \\ \iota_{abort}\sigma & x = \iota_{abort}\sigma \\ \iota_{out}(n, f_*\omega) & x = \iota_{out}(n, \omega) \\ \iota_{in}(f_* \cdot g) & x = \iota_{in} g \end{cases}$$

$$f_+ x = \begin{cases} \bot_\Omega & x = \bot_\Omega \\ \iota_{term}\sigma & x = \iota_{term}\sigma \\ f\sigma & x = \iota_{abort}\sigma \\ \iota_{out}(n, f_+\omega) & x = \iota_{out}(n, \omega) \\ \iota_{in}(f_+ \cdot g) & x = \iota_{in} g \end{cases}$$

$$f_\dagger x = \begin{cases} \bot_\Omega & x = \bot_\Omega \\ \iota_{term}(f\sigma) & x = \iota_{term}\sigma \\ \iota_{abort}(f\sigma) & x = \iota_{abort}\sigma \\ \iota_{out}(n, f_\dagger\omega) & x = \iota_{out}(n, \omega) \\ \iota_{in}(f_\dagger \cdot g) & x = \iota_{in} g \end{cases}$$

$$\begin{aligned} [\![\mathbf{skip}]\!]_\sigma &= \iota_{term}\sigma \\ [\![\mathbf{fail}]\!]_\sigma &= \iota_{abort}\sigma \\ [\![v := e]\!]_\sigma &= \iota_{term}[\sigma|v : [\![e]\!]_\sigma] \\ [\![!e]\!]_\sigma &= \iota_{out}([\![e]\!]_\sigma, \iota_{term}\sigma) \end{aligned}$$

$$
\begin{aligned}
[\![\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1]\!]_\sigma &= \begin{cases} [\![c_0]\!]_\sigma & \text{si } [\![b]\!]_\sigma \\ [\![c_1]\!]_\sigma & \text{si no} \end{cases} \\
[\![c_0; c_1]\!]_\sigma &= [\![c_1]\!]_*([\![c_0]\!]_\sigma) \\
[\![\textbf{catchin } c_0 \textbf{ with } c_1]\!]_\sigma &= [\![c_1]\!]_+([\![c_0]\!]_\sigma) \\
[\![\textbf{newvar } \mathsf{v} := \mathsf{e} \textbf{ in } \mathsf{c}]\!]_\sigma &= (\lambda\sigma' \in \Sigma.\ [\sigma' | v : \sigma\ v])_\dagger([\![c]\!][\sigma | v : [\![e]\!]_\sigma]) \\
[\![\textbf{while } b \textbf{ do } c]\!] &= \textstyle\bigsqcup_{i=0}^{\infty} F^i\ \bot_{\Sigma \to \Omega}
\end{aligned}
$$

donde

$$
F\ w\ \sigma = \begin{cases} w_*([\![c]\!]_\sigma) & \text{si } [\![b]\!]_\sigma \\ \iota_{term}\sigma & \text{si no} \end{cases}
$$

13. Semántica Operacional

La semántica operacional de paso pequeño (small-step) define la ejecución de un programa como una transición entre configuraciones.

Definimos los conjuntos de configuraciones y la relación de transición $\to$ de la siguiente manera:

- **Configuraciones terminales:** $\Gamma_T = \Sigma \cup (\{\textbf{abort}\} \times \Sigma)$
- **Configuraciones no-terminales:** $\Gamma_{NT} = \langle comm \rangle \times \Sigma$
- **Conjunto de configuraciones:** $\Gamma = \Gamma_T \cup \Gamma_{NT}$
- **Relación de un paso:** $\to \subseteq \Gamma_{NT} \times \Gamma$

A continuación, definimos la relación $\to$ inductivamente mediante las siguientes reglas:

Comandos Atómicos

$$
(\text{skip})\frac{}{\langle \textbf{skip}, \sigma \rangle \to \sigma} \qquad (\text{assign})\frac{}{\langle v := e, \sigma \rangle \to [\sigma | v : [\![e]\!]\sigma]}
$$

$$
(\text{fail})\frac{}{\langle \textbf{fail}, \sigma \rangle \to \langle \textbf{abort}, \sigma \rangle}
$$

Composición Secuencial

$$
(\text{seq-ok})\frac{\langle c_0, \sigma \rangle \to \sigma'}{\langle c_0; c_1, \sigma \rangle \to \langle c_1, \sigma' \rangle}
$$

$$
(\text{seq-ab})\frac{\langle c_0, \sigma \rangle \to \langle \textbf{abort}, \sigma' \rangle}{\langle c_0; c_1, \sigma \rangle \to \langle \textbf{abort}, \sigma' \rangle}
$$

$$
(\text{seq-nt})\frac{\langle c_0, \sigma \rangle \to \langle c_0', \sigma' \rangle}{\langle c_0; c_1, \sigma \rangle \to \langle c_0'; c_1, \sigma' \rangle}
$$

Condicional

$$\text{(if-true)}\frac{\text{si } [\![b]\!]\sigma = T}{\langle \textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1, \sigma\rangle \to \langle c_0, \sigma\rangle}$$

$$\text{(if-false)}\frac{\text{si } [\![b]\!]\sigma = F}{\langle \textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1, \sigma\rangle \to \langle c_1, \sigma\rangle}$$

Ciclo While

$$\text{(while-false)}\frac{\text{si } [\![b]\!]\sigma = F}{\langle \textbf{while } b \textbf{ do } c, \sigma\rangle \to \sigma}$$

$$\text{(while-true)}\frac{\text{si } [\![b]\!]\sigma = T}{\langle \textbf{while } b \textbf{ do } c, \sigma\rangle \to \langle c; \textbf{while } b \textbf{ do } c, \sigma\rangle}$$

Variables Locales

$$\text{(loc-ok)}\frac{\langle c, [\sigma|v : [\![e]\!]\sigma]\rangle \to \sigma'}{\langle \textbf{newvar } v := e \textbf{ in } c, \sigma\rangle \to [\sigma'|v : \sigma v]}$$

$$\text{(loc-abort)}\frac{\langle c, [\sigma|v : [\![e]\!]\sigma]\rangle \to \langle \textbf{abort}, \sigma'\rangle}{\langle \textbf{newvar } v := e \textbf{ in } c, \sigma\rangle \to \langle \textbf{abort}, [\sigma'|v : \sigma v]\rangle}$$

$$\text{(loc-nt)}\frac{\langle c, [\sigma|v : [\![e]\!]\sigma]\rangle \to \langle c', \sigma'\rangle}{\langle \textbf{newvar } v := e \textbf{ in } c, \sigma\rangle \to \langle \textbf{newvar } v := \sigma' v \textbf{ in } c', [\sigma'|v : \sigma v]\rangle}$$

Captura de Fallas

$$\text{(catch-ok)}\frac{\langle c, \sigma\rangle \to \sigma'}{\langle \textbf{catchin } c \textbf{ with } \hat{c}, \sigma\rangle \to \sigma'}$$

$$\text{(catch-abort)}\frac{\langle c, \sigma\rangle \to \langle \textbf{abort}, \sigma'\rangle}{\langle \textbf{catchin } c \textbf{ with } \hat{c}, \sigma\rangle \to \langle \hat{c}, \sigma'\rangle}$$

$$\text{(catch-nt)}\frac{\langle c, \sigma\rangle \to \langle c', \sigma'\rangle}{\langle \textbf{catchin } c \textbf{ with } \hat{c}, \sigma\rangle \to \langle \textbf{catchin } c' \textbf{ with } \hat{c}, \sigma'\rangle}$$

13.1. Propiedades de las Ejecuciones

- **Progreso:** Para todo $\langle c, \sigma\rangle$ existe γ tal que $\langle c, \sigma\rangle \to \gamma$.
- **Determinismo:** Si $\langle c, \sigma\rangle \to \gamma$ y $\langle c, \sigma\rangle \to \gamma'$, entonces $\gamma = \gamma'$.
- **Función:** La relación $\to$ define una función de Γ_{NT} a Γ.

13.2. Ejecución Formal

Definimos $\rightarrow^*$ como la clausura reflexiva y transitiva de $\rightarrow$:

$$(\text{refl})\frac{}{\gamma \rightarrow^* \gamma} \qquad (\text{paso})\frac{\gamma \rightarrow \gamma'}{\gamma \rightarrow^* \gamma'}$$

$$(\text{trans})\frac{\gamma_1 \rightarrow^* \gamma_2 \quad \gamma_2 \rightarrow^* \gamma_3}{\gamma_1 \rightarrow^* \gamma_3}$$

Una ejecución es una secuencia maximal de transiciones. Es decir, $\rho : \gamma_0 \rightarrow^* \gamma_n$ es maximal si γ_n es una configuración terminal o si ρ es infinita.

Si ρ es una secuencia infinita decimos que diverge y escribimos $\gamma \uparrow$. Definimos una nueva semántica usando la ejecución:

$$\{\![_]\!\} : \langle comm \rangle \rightarrow \Sigma \rightarrow \Sigma'$$

$$\{\![c]\!\}\sigma = \begin{cases} \bot & \text{si } \langle c, \sigma \rangle \uparrow \\ \gamma & \text{si } \langle c, \sigma \rangle \rightarrow^* \gamma \in \Gamma_T \end{cases}$$